

Finite-depth Lyapunov-growth estimation in GNNs

Finite-depth Lyapunov-growth estimation in GNNs

Status: reproducible pilot study, 2026-09-24.

Question: how should we estimate the top depth-wise Jacobian growth rate of a trained GNN, and how dependable are random tangent probes across GNNs and graph/data conditions?

This report concerns the finite network quantity

$$\lambda_K^{\text{top}}(G, x; \Phi) = \frac{1}{K} \log \sigma_1(B_K), \quad B_K = D_{h_0}(F_{K-1,G} \circ \dots \circ F_{0,G})(h_0),$$

where h_0 is the output of the encoder and h_K the output of the last hidden propagation layer. It does **not** include encoder or decoder derivatives and is not automatically an infinite-depth Lyapunov exponent. All results use the complete layer derivative at the forward state, including activation and state-conditioned sheaf restriction-map derivatives.

Primary-estimand implementation detail: no tangent vector is mean-centered or projected between layers when estimating B_K . Pierre’s P -projected procedure measures a different product, so it is evaluated and labeled separately below. Scalar norm renormalization is used only to prevent numerical overflow/underflow.

Executive conclusions

1. **A maximum over independent JVP probes is correct but was not useful as a point estimate here.** It is a rigorous lower bound for every finite M and converges almost surely as $M \rightarrow \infty$. Yet even **1,024** probes did not meet the predeclared 0.01-per-layer error target in any of the 432 trained calibration streams: pooled median gaps remained 0.395 for GCN and 0.109 for the deep sheaf baseline.
2. **Matrix-free power iteration on $B_K^* B_K$ is the right default for the top finite-depth rate when VJPs are available.** With three starts and 160 iterations, all 36 tiny trained calibration cells met residual $\leq 10^{-7}$ and agreed with reverse-mode dense-SVD references to mean absolute rate error 3.9×10^{-17} (maximum 2.2×10^{-16}).
3. **Probe count is architecture/spectrum dependent, not a universal GNN hyperparameter.** In this study, random probes were looser for the depth-3 GCN and depth-3 ChebNet than for the depth-10 sheaf baseline despite comparable tangent dimensions; graph family had a smaller effect than architecture. Training did not make random probes reliably tighter in the initialization ablation.
4. **Do not infer oversquashing or task utility from this top rate alone.** The GCN and sheaf baseline have positive top hidden-state rates here while their test accuracy differs sharply under a heterophilic distribution shift. Top global sensitivity does not identify source-to-target communication or spatial delocalization.
5. **Pierre’s mean-zero projection gives a smaller, deliberately different observable.** Matched projected-cocycle exact rates averaged 0.344 rather than 0.674 for GCN, 0.427 rather than 0.523 for sheaf, and 0.844 rather than 1.112 for ChebNet. This supports the interpretation that a full-space top rate can be elevated by consensus-like modes, while also showing that non-constant modes still grow on average in these trained examples.

The findings are strong evidence for estimator choice on these controlled models, not a claim that all GNNs or real-world datasets behave identically. Broader datasets, architectures and hyperparameter budgets are the next research phase.

Estimators: what each algorithm estimates

Algorithm	Quantity	What is rigorous	Practical verdict
Dense Jacobian + SVD	Exact λ_K^{top}	Exact finite- K reference	Use only at tiny tangent dimension
M full-space JVP probes	$\hat{\lambda}_{K,M}$	Finite- M lower bound; a.s. convergence as $M \rightarrow \infty$	Use as a lower-bound/spectral-shape diagnostic, not the default point estimate
JVP/VJP power on $B_K^* B_K$	Approximation to λ_K^{top}	Standard power convergence conditional on start overlap and spectral separation; final residual is directly checkable	Default top-rate estimator
Lanczos/Krylov on $B_K^* B_K$	Approximation to the same top singular value	Matrix-free extremal-eigenvalue method	Escalate to it for poor power residuals or clustered top spectrum
QR/Benettin frame	Long-depth Lyapunov spectrum under an extendable cocycle	Standard numerical asymptotic-spectrum method	Not interchangeable with finite- K singular values
Layerwise singular norms/product bounds	An upper bound or a different diagnostic	Not $\sigma_1(B_K)$ in general	Never report as this exponent
Finite nonlinear perturbations	Secant growth	Depends on perturbation scale and activation region	Optional autodiff audit only

Here $\hat{\lambda}_{K,M} = \max_{m \leq M} K^{-1} \log \|B_K q_m\|$. The key random-probe obstruction is visible in the rank-one case. If $B = \sigma uv^\top$ and q is uniform on S^{D-1} , then $\|Bq\|/\sigma = |v^\top q|$ and $(v^\top q)^2 \sim \text{Beta}(1/2, (D-1)/2)$. Typical overlap is $O(D^{-1/2})$. Thus a narrow leading direction can require very many parallel probes even though a power step aligns it immediately. A broad near-top singular subspace produces the opposite behavior. This is why a probe budget cannot be fixed from depth alone.

Controlled GNN architectures

Exact equations and scope are in notes/experiment-baselines.md. We used:

- **GCN:** a depth-3 Kipf-Welling stack, $h_{\ell+1} = \text{ReLU}(\tilde{A}h_\ell W_\ell)$, with self-looped symmetric normalized adjacency.
- **ChebNet:** depth 3 and Chebyshev order 3, with the self-loop-free symmetric normalized Laplacian and $\lambda_{\max} = 2$ scaling; it supplies a polynomial-filter contrast to one-hop GCN propagation.
- **Deep feature-conditioned diagonal-sheaf baseline:** depth 10, endpoint-dependent diagonal restriction maps, an assembled sheaf-Laplacian action and a residual ReLU update. It is inspired by Bodnar et al.’s neural sheaf diffusion, but is **not** claimed to reproduce every detail of that paper. Its implemented normalization is one-sided row normalization, not symmetric $D^{-1/2} L_{\mathcal{F}} D^{-1/2}$.

Datasets, splits, and the input-dependent rate

Graphs, features, and labels

This is a supervised node-classification pilot using deterministic synthetic graph generators. Every feature matrix has four columns. For node label $y_i \in \{0, 1\}$, the input signal is

$$x_i = (2y_i - 1)e_1 + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I_4).$$

Thus the first coordinate is a noisy class signal and the other three coordinates are noise; the configured noise standard deviation is 1.0. All node labels are used in the graph-level cross-entropy training loss.

- **Homophilic contextual SBM:** 12 nodes, with six nodes in each class. Undirected edges are sampled independently in the upper triangle with within-class probability $p_{\text{same}} = 0.75$ and between-class probability $p_{\text{other}} = 0.10$.
- **Heterophilic contextual SBM:** the same 12-node balanced classes and feature model, but the probabilities are reversed: $p_{\text{same}} = 0.10$ and $p_{\text{other}} = 0.75$. This is a structural distribution shift; it is never used for training or hyperparameter selection.
- **Barbell:** two six-node cliques joined by three intermediate path nodes, hence 15 nodes total. Labels divide the graph into left and right sides, with the class boundary inside the bridge. Features use the same noisy class signal. This is a bottleneck/topology shift and is also test-only.

The stored adjacencies contain no self-loops. Each architecture then applies its documented normalization: the GCN adds self-loops before symmetric normalization, whereas ChebNet uses the self-loop-free normalized Laplacian. The sheaf baseline constructs its own learned sheaf operator from the stored graph.

Training, validation, testing, and calibration

The split consists of **16 homophilic training graphs, 6 independently generated homophilic validation graphs, and 12 independently generated test graphs for each of the three conditions** (36 test graphs total). The base data seed is 101: training uses seeds 101–116, validation 10101–10106, and the homophilic, heterophilic, and barbell test generators start at 20101, 20102, and 20103 respectively. Exact per-example seeds and generator conditions are recoverable from the committed config and manifest. The generated examples are fixed across architectures and weight-initialization seeds.

Each hyperparameter candidate is trained for 120 full-graph Adam epochs with weight decay 10^{-4} ; the loss is the mean of the 16 per-graph node-classification losses. For each of two model seeds (11 and 29), validation accuracy alone selects hidden width from $\{8, 12\}$ and learning rate from $\{0.003, 0.01\}$. Test data do not enter model or hyperparameter selection. Reported test accuracy uses all 12 graphs in each condition. Jacobian calibration is more expensive and therefore uses the first **three** test examples per condition, giving $2 \text{ model seeds} \times 3 \text{ conditions} \times 3 \text{ examples} = \mathbf{18 \text{ pointwise calibration cells per architecture}}$. Each unprojected cell has an independent dense reference, a residual-checked power estimate, and 12 nested random-probe streams.

What varies when $\lambda_K^{\text{top}}(G, x; \Phi)$ is reported

The rate is evaluated **pointwise**. Within one calibration cell, the graph G , input feature signal x , trained parameters Φ , stochastic state, and resulting forward trajectory $h_0, \dots, h_K$ are fixed. The random vectors q_m sample tangent directions at that fixed h_0 ; they are not new data inputs. Every J_ℓ is evaluated along that input-specific trajectory. Consequently, the almost-sure $M \rightarrow \infty$ statement for random probes and the dense/power comparison both hold separately for each fixed (G, x, Φ) .

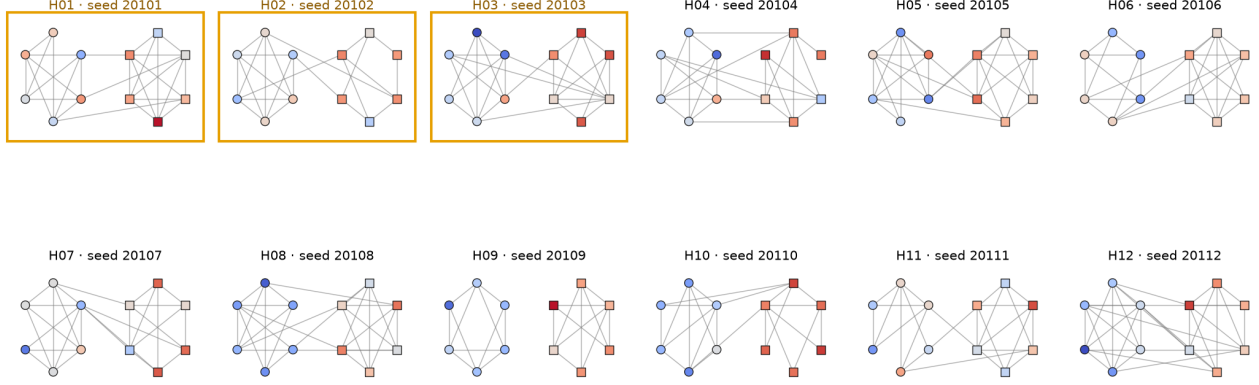
Across cells, the report treats $\lambda_K^{\text{top}}(G, x; \Phi)$ as an empirical random variable: boxplots and aggregate gaps pool the pointwise values from held-out examples, graph conditions, and the two trained model seeds. This pilot does **not** isolate dependence on x at fixed G : each synthetic draw generates the graph and its feature noise jointly, so within-condition spread conflates graph-to-graph variation, input-signal variation, and model-seed variation. With only three calibration examples per condition and seed, the displayed distributions are descriptive rather than population-level estimates. Also, h_0 is the encoded input and the reported derivative excludes encoder/readout derivatives; it is not the end-to-end $D_x \Phi_G(x)$.

The 36 held-out test graph/signal examples

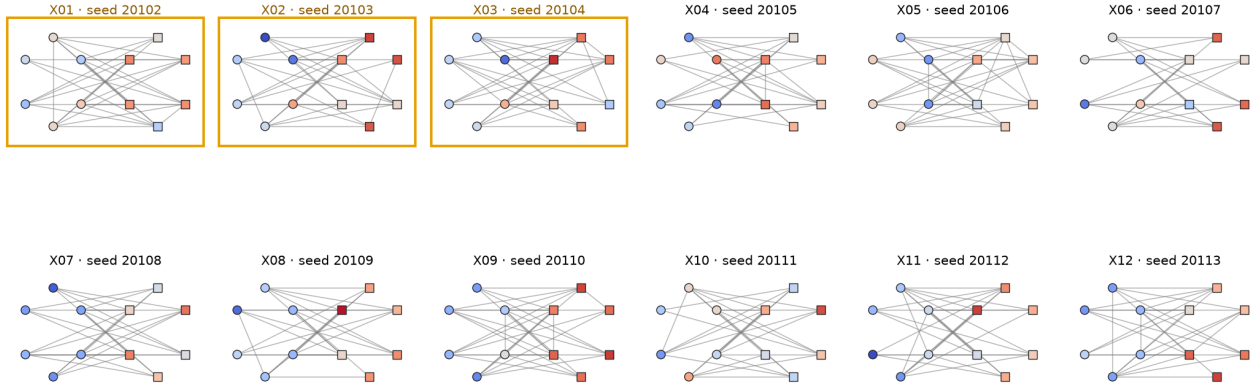
The complete split has 58 unique graphs: 16 training, 6 validation, and 36 testing. The next page shows all **36 test graphs**. Colour gives x_{i1} ; circles/squares give labels 0/1; the other three Gaussian-noise coordinates are omitted. Gold frames mark the **nine unique (G, x) calibration examples** (first three per family), reused across model seeds and architectures.

All 36 held-out test graph/signal examples

Homophilic SBM



Heterophilic SBM



Barbell

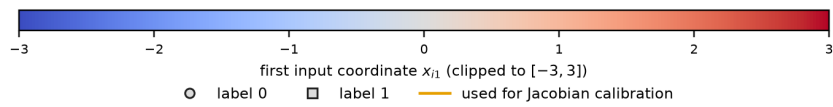
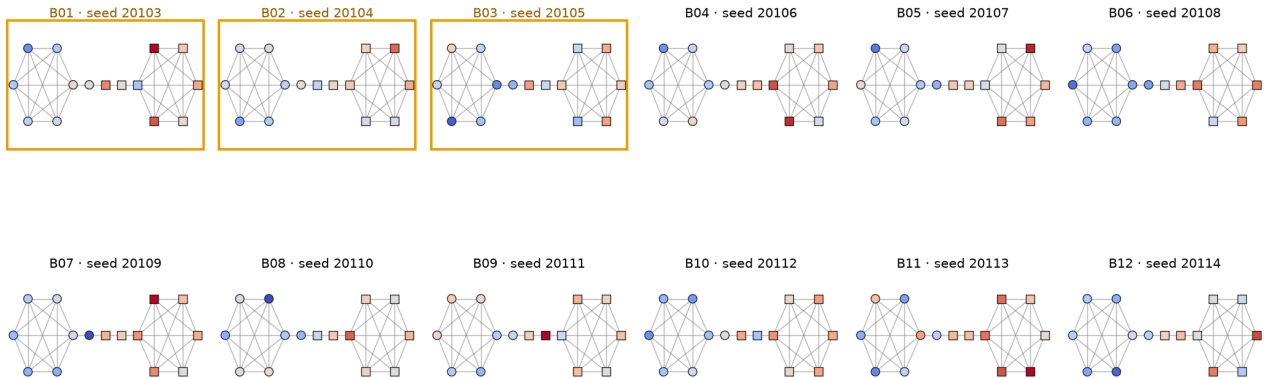


Figure 1: All held-out test graph/signal examples

All three models learned the in-distribution task comparably (mean held-out homophilic accuracy: GCN 0.865; ChebNet 0.861; sheaf 0.878). Under the heterophilic shift, ChebNet scored 0.524, the sheaf baseline 0.469, and GCN 0.312; all were high on the barbell condition (0.964, 0.958 and 0.944 respectively). These accuracy values validate that the models were not merely untrained random maps, but they are not a broad architecture benchmark.

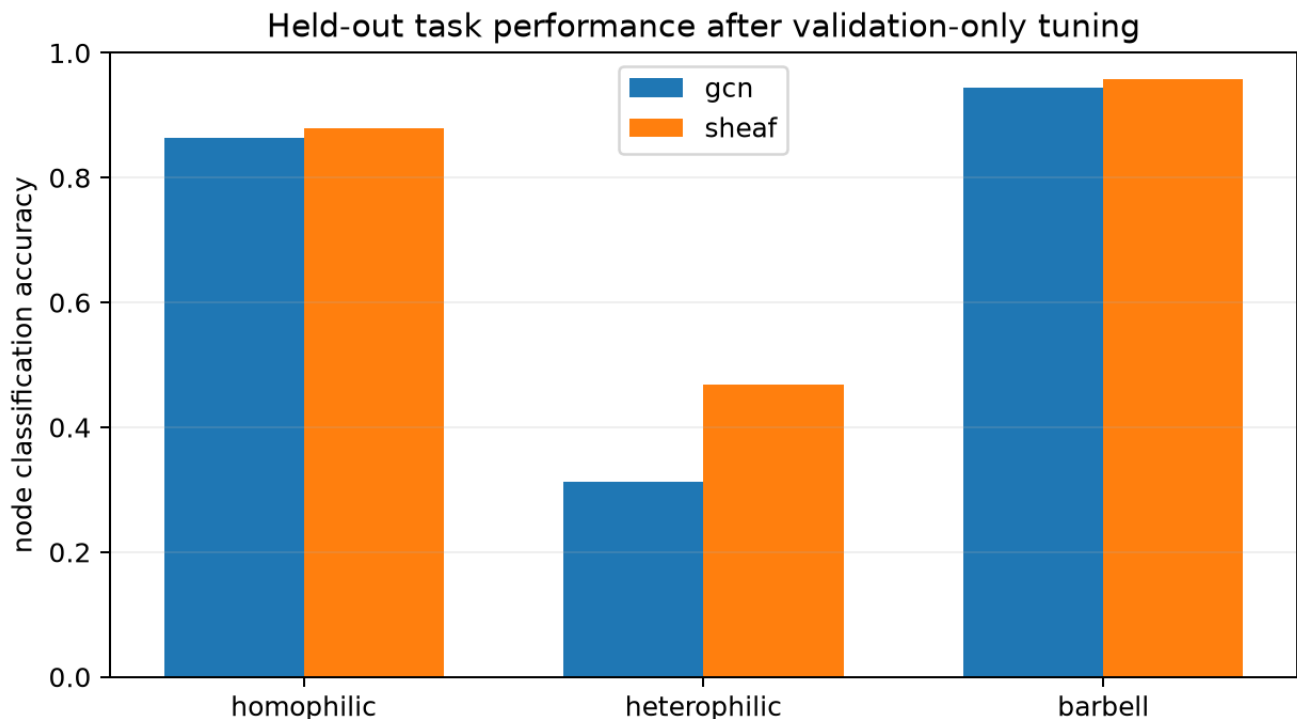


Figure 2: Held-out task accuracy

Verification before measurement

The implementation has Slurm-verified unit tests at clean commit c909dd60d39d51e90ad15413af8470b0da4735fd (job 22347; 8 passed; evidence in RESEARCH_LOG.md):

- explicit linear product equals the JVP-built product;
- zero product returns $-\infty$;
- an actual regular-graph GCN constant mode survives (detecting an accidental P projection);
- non-unit supplied probes are rejected;
- power agrees with dense SVD on a known linear product;
- nonlinear GCN and sheaf JVP products agree with an **independent reverse-mode dense autodiff Jacobian**. The declared ReLU convention is PyTorch’s derivative zero at a kink.

For every small experimental cell, the maximum entrywise discrepancy between JVP-built and reverse-mode product was at most 2.2×10^{-16} for GCN and 2.1×10^{-14} for sheaf. This validates autodiff implementations, but not a hypothetical shared framework bug; it is why both a structural test suite and two AD modes are retained.

Experience 1: random probes converge far too slowly here

The v2 study used 12 independent nested streams through each trained cell, with $M = 1, 2, 4, \dots, 1024$. Shading is the 5th–95th percentile over all selected seeds, trajectories and streams within a graph family.

Random-probe lower-bound convergence by architecture and graph family

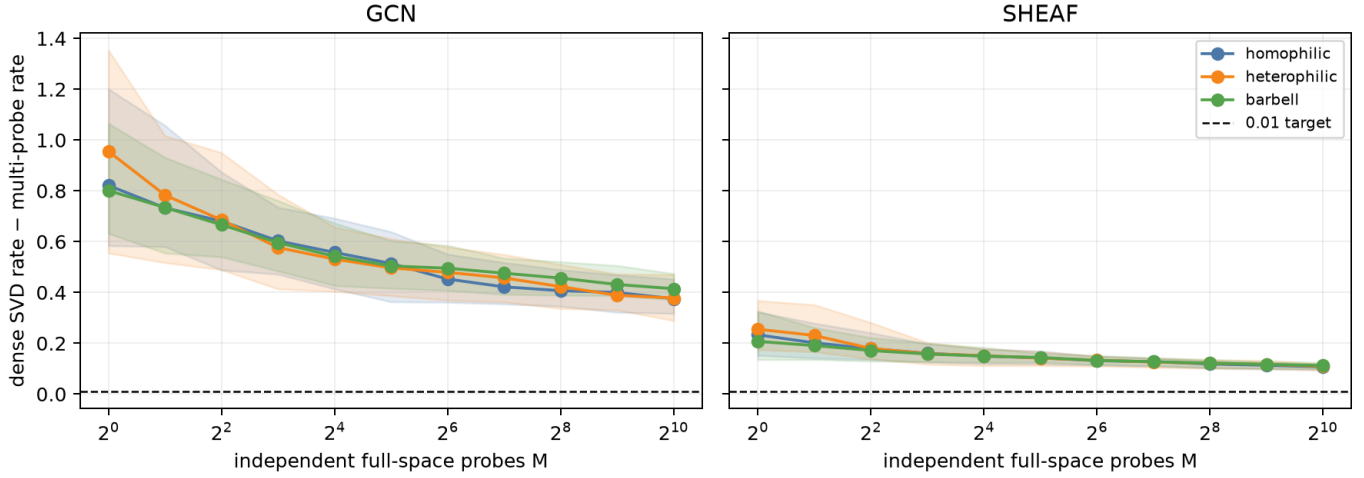
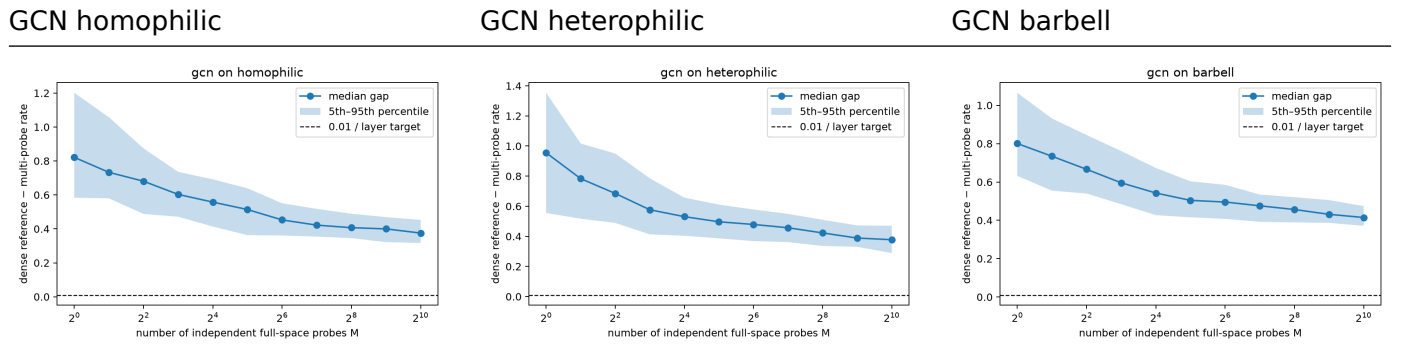


Figure 3: Probe convergence summary

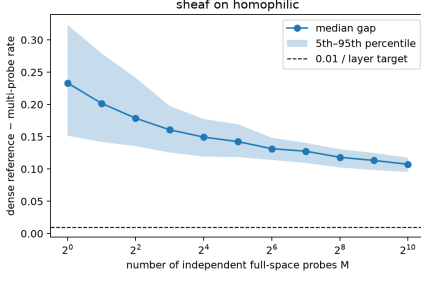
At $M = 1$, pooled median gap was 0.847 (GCN) and 0.231 (sheaf). At $M = 128$, it was 0.449 and 0.127. At $M = 1024$, it was 0.395 and 0.109. The target was 0.01 per layer, so the relevant result is not merely that convergence is slow: **coverage at the target was 0/72 for every model-dataset family at $M = 1024$.**

Model	Dataset family	Median gap at $M = 1024$	95th percentile gap	Target coverage
GCN	homophilic	0.376	0.452	0/72
GCN	heterophilic	0.379	0.469	0/72
GCN	barbell	0.417	0.473	0/72
Sheaf	homophilic	0.107	0.117	0/72
Sheaf	heterophilic	0.109	0.119	0/72
Sheaf	barbell	0.112	0.124	0/72

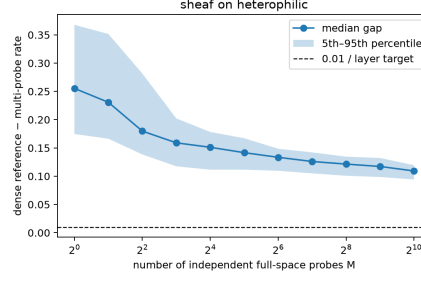
The per-family plots make clear that dataset family changes the curve somewhat, but the architecture-level difference is larger in this pilot:



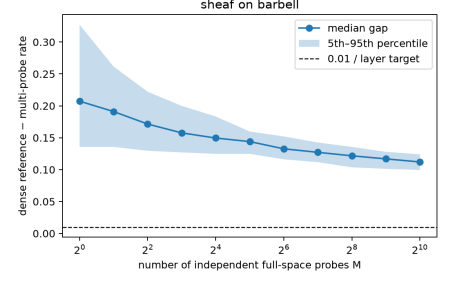
Sheaf homophilic



Sheaf heterophilic



Sheaf barbell

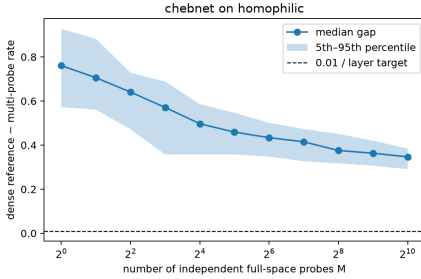


Interpretation: these data falsify the initially tempting default “use 128 random probes and call the maximum the exponent.” They do not say that a random-probe lower bound is wrong; it is doing exactly what it promises. The high gap suggests that the dominant finite-product direction is fairly concentrated relative to random full-space directions, especially for these trained GCNs and ChebNets.

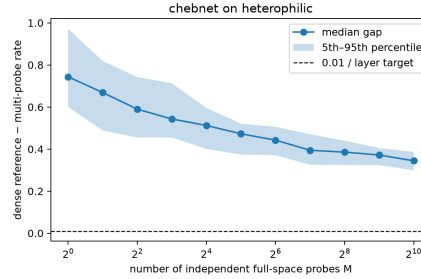
ChebNet replication

A separately tuned depth-3/order-3 ChebNet calibration (two seeds, three trajectories per graph family, 12 streams through 1024 probes) replicates the random-probe conclusion: median $M = 1024$ gaps were 0.347 (homophilic), 0.346 (heterophilic), and 0.376 (barbell), with 0/72 streams at the 0.01 target in every family. All 18 corresponding three-start/160-iteration power cells met the residual criterion. Its curve lies between the GCN and sheaf cases, showing that the result is neither specific to a one-hop filter nor explained only by depth.

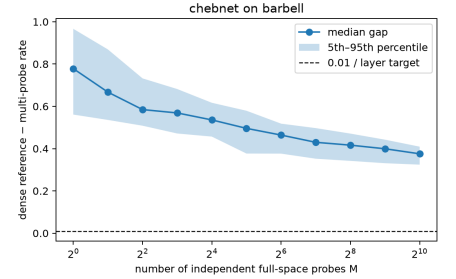
ChebNet homophilic



ChebNet heterophilic



ChebNet barbell



Experience 2: power iteration succeeds on the same finite products

For the same 36 trained tiny cells, three random-start power runs of $160 B_K^* B_K$ iterations were performed. The best returned rate was accepted only when its explicit relative residual was $\leq 10^{-7}$. All 36 cells met the condition. Compared with the reverse-mode dense SVD rate, the mean absolute error was 3.9×10^{-17} and maximum 2.2×10^{-16} ; these are tiny-reference agreement results, not an assertion that 160 iterations will suffice at every scale.

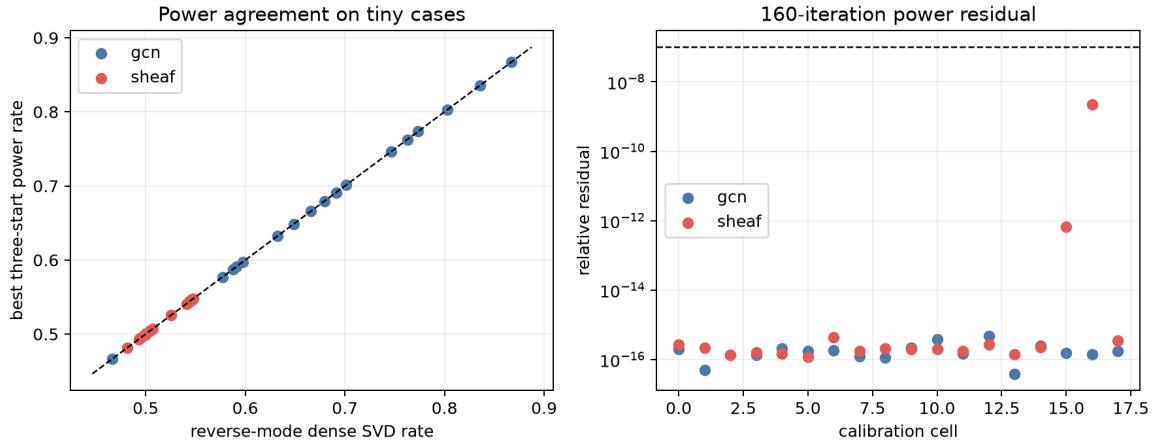


Figure 4: Power versus exact SVD

This contrast is expected: power iteration actively rotates a generic nonzero-overlap start toward the dominant right singular direction, whereas independent raw probes only wait to sample a close one. The final residual is essential. In v1, a single 80-iteration start failed the 10^{-7} residual criterion in two deep-sheaf/barbell cells; v2's three-start/160-iteration protocol resolved all tiny cells. A low residual validates convergence of the chosen Krylov iterate, while agreement of multiple starts helps reveal a poor start or numerical pathology.

Experience 3: rates, graph families and training state

The exact top rate is positive in this pilot. That means selected hidden perturbations can grow across depth; it does not by itself predict prediction accuracy or distant-node sensitivity.

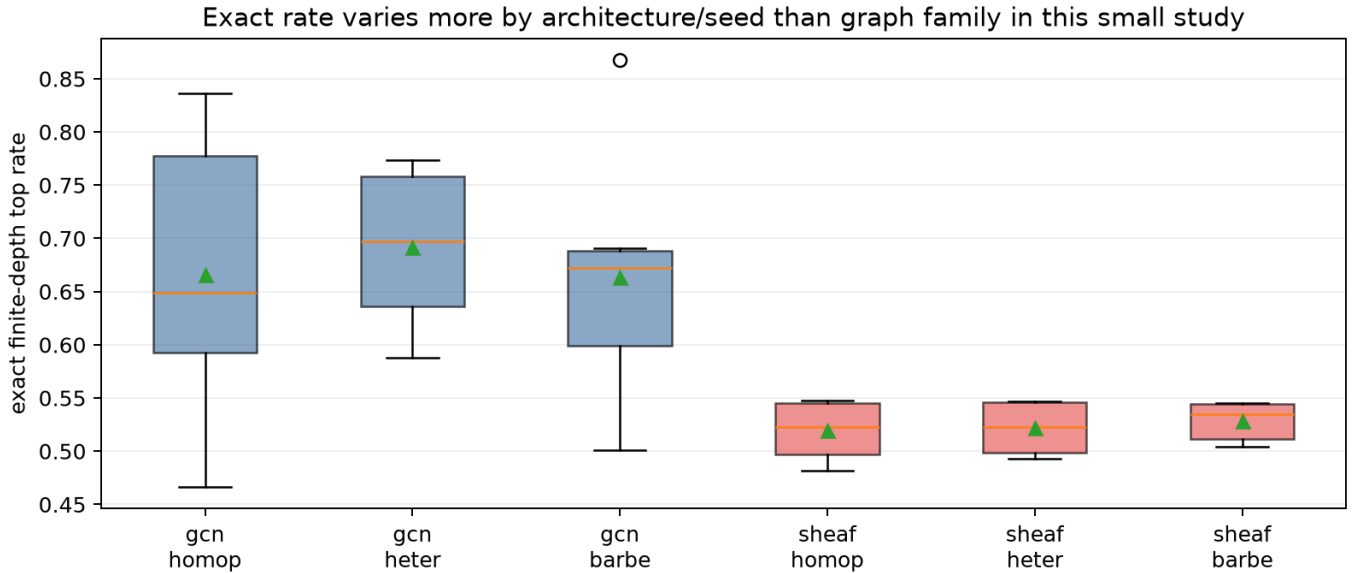


Figure 5: Exact-rate distribution

A separate trained-versus-untrained ablation used two seeds, three trajectories per family, eight streams and up to 512 probes. Training did not make raw probe estimates tight. At $M = 512$, the GCN median gaps ranged from 0.418–0.430 trained and 0.396–0.418 untrained; sheaf gaps ranged from 0.111–0.116 trained and 0.111–0.114 untrained. This is useful negative evidence: random-probe behavior is not safely inferred from whether a model has learned the task.

Training changes the product spectrum and random-probe convergence

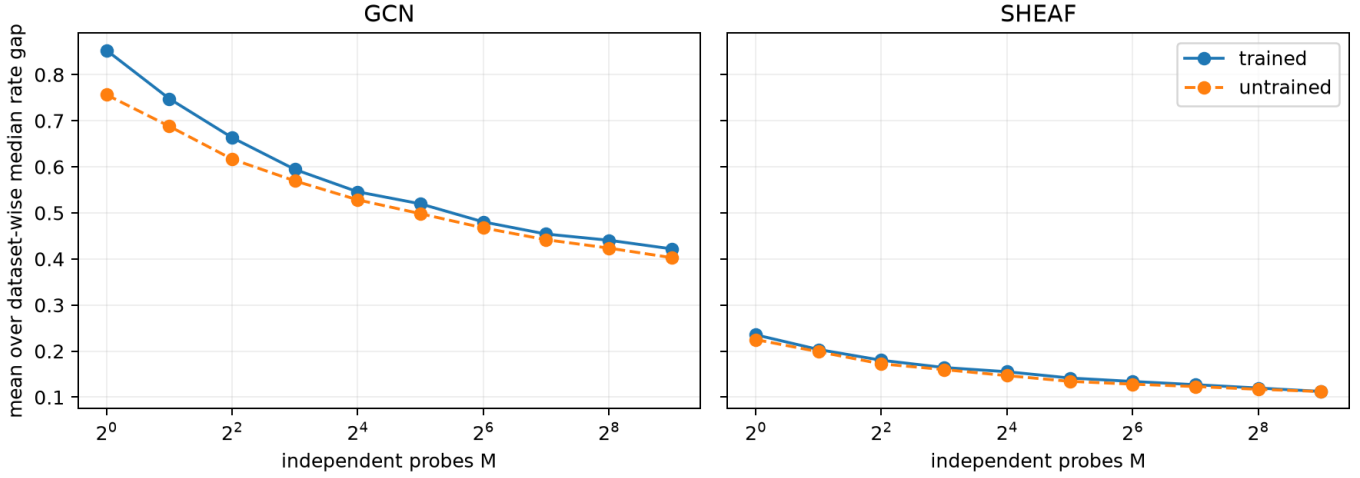


Figure 6: Training-state probe comparison

Experience 4: estimator wall-clock cost

Wall-clock time was measured for methods targeting the same **unprojected** product B_K . The benchmark used the first test trajectory from each graph family, both model seeds, and all three architectures: six fixed (G, x, Φ) cells per architecture. Each method received one warm-up and three timed repeats, giving 18 timings per table entry. PyTorch float64 ran eagerly on one CPU thread in Slurm job 22388. The timed region starts after training, forward-state construction, layer-function setup, and probe generation; it includes only the estimator/solver call.

Median seconds per fixed cell were:

Model	Dense Jacobian + SVD	Probes $M = 128$	Probes $M = 1024$	Power 3×160
GCN	0.0033	0.0038	0.0074	1.690
ChebNet	0.0045	0.0075	0.0138	3.510
Sheaf	0.071	0.103	0.293	38.46

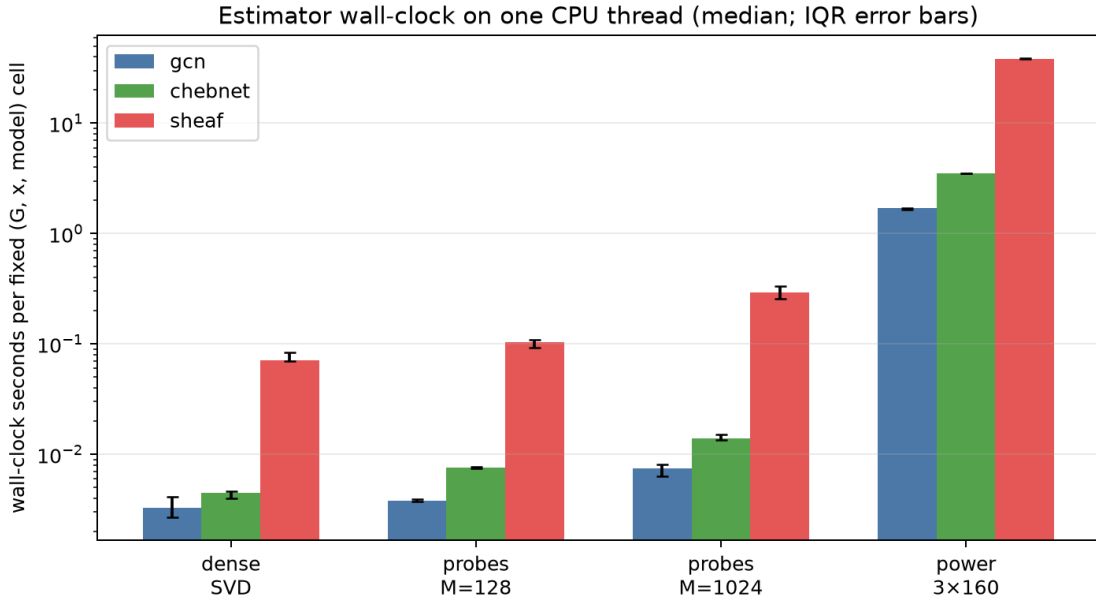


Figure 7: Estimator wall-clock comparison

On these deliberately tiny tangent spaces, dense SVD is both exact and fastest. That is why the recommended protocol uses it for calibration whenever the explicit Jacobian fits. Vectorized random probes are also cheap here, but their severe accuracy gaps remain; low wall-clock cost does not make them good point estimates. The residual-checked power protocol is much slower in this eager CPU implementation because three starts at 160 iterations require 480 forward/adjoint product iterations, especially through the depth-10 state-adaptive sheaf. All 54 timed power repeats converged, with maximum relative residual 4.0×10^{-12} .

This timing result does **not** overturn the scalable recommendation. Dense Jacobian storage grows quadratically with tangent dimension and eventually becomes infeasible, whereas matrix-free power avoids materializing B_K . Conversely, the vectorized $M = 1024$ timing uses memory proportional to the probe batch. Absolute timings and crossover points will change with dimension, hardware, compilation, batching, and a Lanczos implementation; this is a transparent small-CPU benchmark, not a universal speed ranking.

Experience 5: exact Lyapunov-rate histograms across models and datasets

For each architecture we load a trained checkpoint Φ (depth-3 GCN, depth-3/order-3 ChebNet, depth-10 sheaf; each 0.986 validation accuracy) and draw **the same** 300 graph/signal pairs per family (one shared (G, x) distribution, seeds offset identically for all three models). For each pair the exact reverse-mode dense Jacobian+SVD rate $\lambda_K(G, x; \Phi)$ is computed, giving 900 pointwise samples per model whose empirical distributions are plotted below.

Exact Lyapunov-rate histograms over random (G, x) pairs (same 300 draws/family per model)

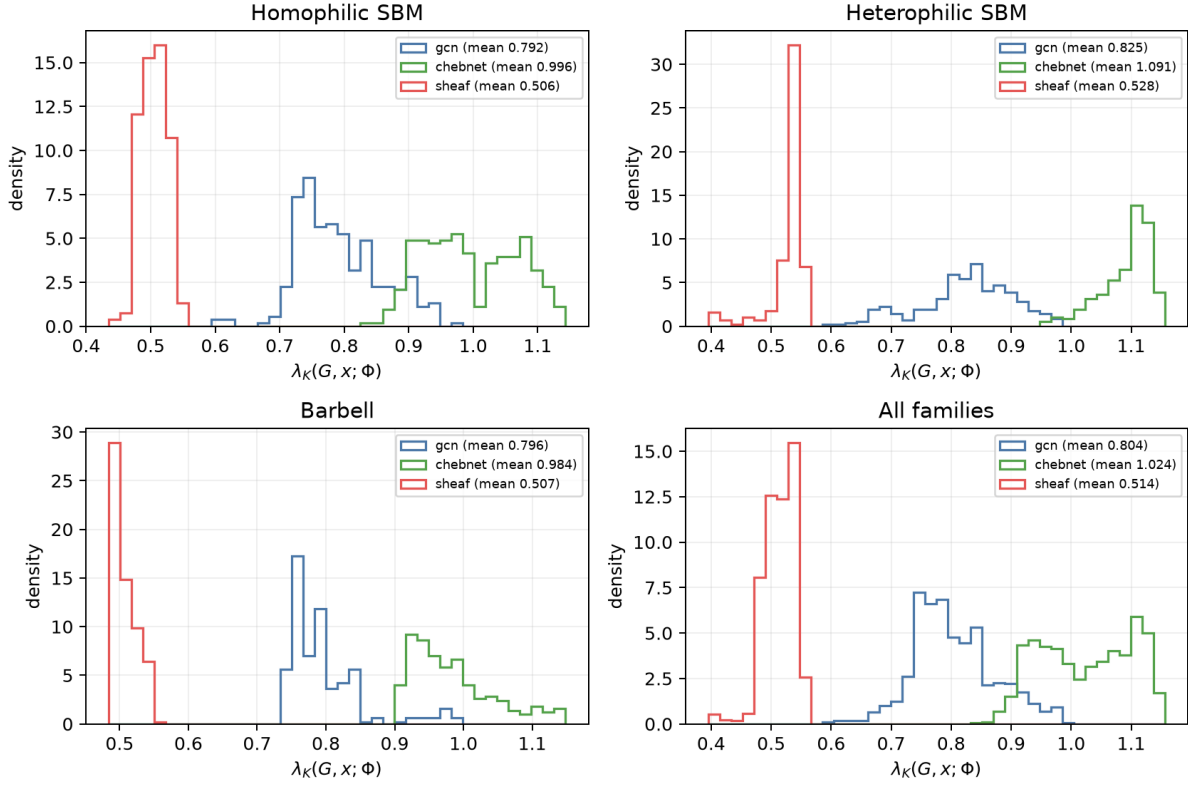


Figure 8: Exact Lyapunov-rate histograms over random (G, x) pairs

Mean exact rates (with median and standard deviation) over the 900 draws per model:

Model	Overall mean	Overall median	Std	Homophilic	Heterophilic	Barbell
GCN	0.804	0.793	0.068	0.792	0.825	0.796
ChebNet	1.024	1.028	0.076	0.996	1.091	0.984
Sheaf	0.514	0.517	0.026	0.506	0.528	0.507

Three features stand out.

1. **Model ordering is clean and consistent.** ChebNet dominates every family (mean ≈ 0.98 – 1.09), GCN is intermediate (≈ 0.79 – 0.83), and the deep sheaf baseline is lowest (≈ 0.51 – 0.53). The histograms are well separated, so a single top rate meaningfully differentiates these trained architectures rather than being buried in sampling noise.
2. **The distributions are narrow relative to their means** (coefficient of variation $\lesssim 0.09$). Across hundreds of graph/signal draws, $\lambda_K(G, x; \Phi)$ varies by only ~ 0.05 – 0.08 (one standard deviation), so the finite-depth growth rate is fairly insensitive to which specific graph/signal input is chosen within a fixed model—at least on this synthetic family scale.
3. **Family has a smaller effect than model.** Within a model, family means sit within ~ 0.03 for the sheaf and GCN and ~ 0.1 for ChebNet, whereas model-to-model differences exceed 0.29 everywhere. The heterophilic condition tends to raise the mean for all three, consistent with the earlier pointwise calibration; it does not reorder the models.

As throughout this report, these are **finite-depth** rates under the fixed-draw empirical distribution, computed with the exact reference; they are not infinite-depth asymptotic Lyapunov exponents and say nothing alone about oversquashing or task utility.

Separate experiment: Pierre’s projected estimator

Pierre’s proposal removes the node-constant component of every tangent after every layer. Let

$$Pz = z - \frac{1}{N} \mathbf{1}\mathbf{1}^\top z$$

act independently on each feature channel. Mean-zero unit probes are initialized in $\mathbf{1}_N^\perp$, and the measured finite product is

$$B_K^P = PJ_{K-1}PJ_{K-2}\cdots PJ_0|_{\mathbf{1}_N^\perp}.$$

This is a coherent transverse/mean-zero observable, but it is **not an estimator of** the actual hidden-state Jacobian $B_K = J_{K-1}\cdots J_0$: projection removes any path that enters a constant node mode and later leaves it. The caveat is stronger for normalized graph operators and sheaves, whose natural null or harmonic subspaces need not equal the raw node-mean-zero subspace. The experiment implements Pierre’s literal node-mean prescription and reports it under a distinct name.

The projected study repeated the same design with matched data/model seeds and tuning grid for all three architectures: 18 pointwise cells per architecture, 12 nested mean-zero streams through $M = 1024$, a dense SVD of the reduced projected product, and three-start/160-iteration projected power iteration. All **54** projected power cells passed relative residual $\leq 10^{-7}$ and agreed with projected dense SVD to numerical precision.

Mean exact rates over the 18 matched cells per architecture were:

Model	Full Jacobian B_K	Pierre projected B_K^P	Reduction
GCN	0.674	0.344	0.330
Sheaf	0.523	0.427	0.096
ChebNet	1.112	0.844	0.268

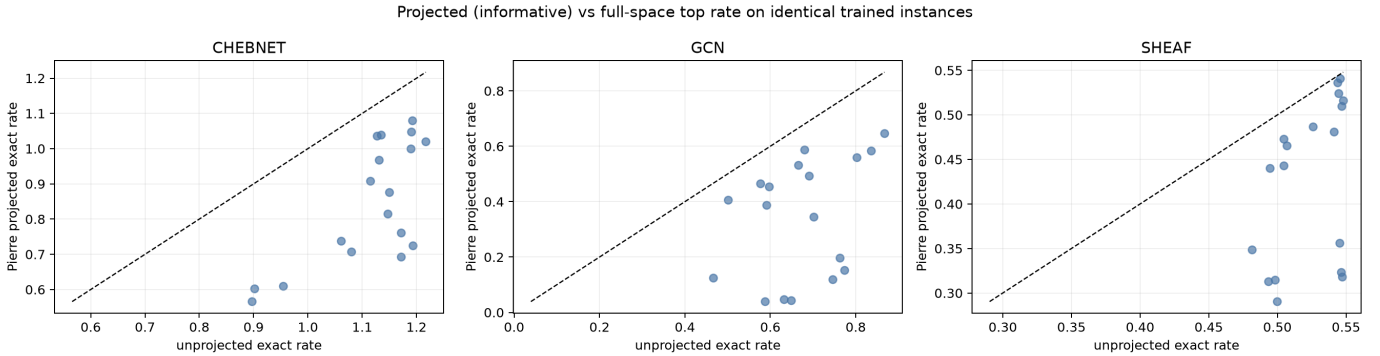


Figure 9: Projected versus unprojected exact rates

Projection therefore lowers the architecture-mean rate in every case, most strongly for GCN. This supports the interpretation that consensus-like directions elevate the full-space top rate. It does **not** make the projected rate generally negative: the projected exact means remain positive, and ChebNet retains substantial non-constant amplification.

The random-probe difficulty also remains. At $M = 1024$, no projected stream reached the 0.01-per-layer target. Comparing the actual finite- M averages—not the exact references—over $18 \text{ cells} \times 12 \text{ streams}$ gives:

Model	Full-space probes	Pierre probes	Pierre minus full
GCN	0.280	-0.038	-0.318

Model	Full-space probes	Pierre probes	Pierre minus full
Sheaf	0.414	0.324	-0.090
ChebNet	0.759	0.517	-0.242

These $M = 1024$ numbers are lower-bound estimates and should not replace the exact/power comparison above. For example, the negative average projected GCN probe result does not establish that its true projected top rate is negative; the projected exact mean is 0.344. It shows how severely random directions can miss the maximizing direction even after mean-zero projection.

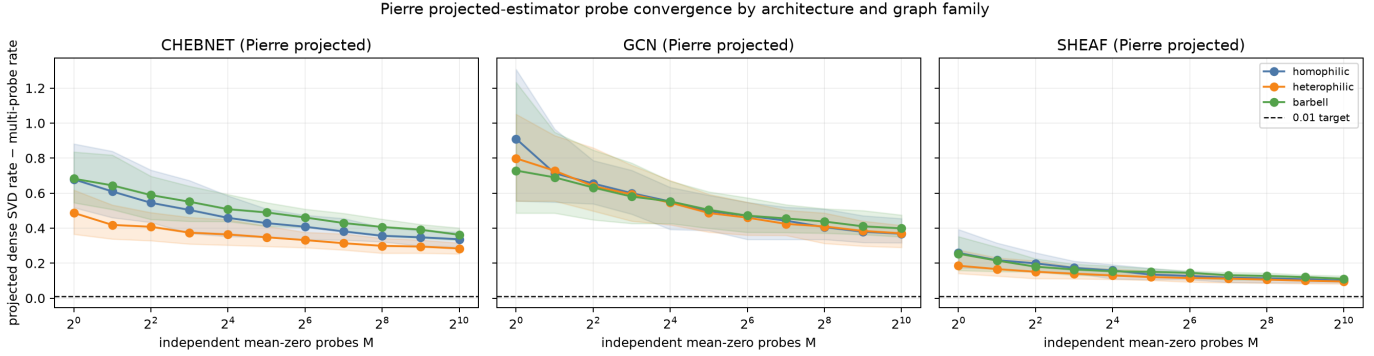


Figure 10: Pierre projected-probe convergence

Full cell-level evidence and caveats are in `notes/pierre-estimator-experiments.md`.

Recommended default protocol

Situation	Recommendation	Required reporting
Tiny calibration, e.g. $D \lesssim 200$	Reverse-mode dense Jacobian/SVD plus JVP-product agreement	Both AD routes, max product discrepancy, exact singular value/rate
Routine finite- K top rate with JVP and VJP	Three-start power iteration on $B_K^* B_K$; 160 iterations initially; accept only residual $\leq 10^{-7}$	each start's rate/residual, accepted start, JVP/VJP sweep count, precision
Power fails residual/start agreement	Run more iterations; then matrix-free Lanczos/Krylov; otherwise label result unresolved	residual history and stop reason; never silently emit a number
JVP-only setting	Report nested multi-probe maxima explicitly as lower bounds; calibrate M against a trusted reference by architecture/dimension	M , stream count/seeds, gap coverage curve; no unqualified "exponent" claim
Long, tied/stationary depth process and spectrum needed	QR/Benettin frame, with an explicitly justified stationary cocycle	transient/renormalization schedule and convergence history
Any GNN	evaluation mode; fixed graph and stochastic masks; full derivative through adaptive geometry/normalization; no P	state boundary, hidden width/depth, dtype, graph/example IDs, full commit and lock hashes

The 160/three-start choice is an evidence-supported **initial default**, not a permanent magic number. At larger dimensions, the residual threshold and repeat-start checks—not the iteration count—should

decide acceptance. Lanczos is the next algorithmic upgrade because it uses the same JVP/VJP primitives but can improve convergence for an unfavorable spectrum.

Limits and next experiments

- The graphs are deliberately small because exact SVD is central to calibration. We have not established a power-iteration residual threshold on large real-world GNNs.
- The sheaf architecture is an explicit diagnostic baseline, not a faithful reproduction claim for every Neural Sheaf Diffusion variant. The state-dependent- μ GNN remains underspecified in notes/state-dependent-mu-diffusion-GNN.md.
- The study is node classification on synthetic graph distributions; it does not yet cover citation/coauthor/molecular benchmarks, graph-level tasks, dropout/DropEdge or heterogeneous feature sizes.
- The calibration varies (G, x) jointly and has only three examples per graph condition and model seed. It therefore does not estimate input-only variation $x \mapsto \lambda_K(G, x; \Phi)$ on a fixed graph, nor a high-precision population distribution.
- Global $\sigma_1(B_K)$ cannot diagnose particular source-target blocks of the Jacobian, delocalization, over-smoothing or oversquashing. Those need separate controlled observables.
- The next scale-up should retain this exact-SVD calibration suite, then test power/Lanczos residual behavior and rate distributions on tuned GCN, ChebNet, a specified sheaf implementation, and the specified adaptive- μ model on real benchmarks. Hyperparameters must be selected from validation data only.

Evidence and reproducibility

Raw outputs and tracked evidence use results/<run-id>/ and evidence/<run-id>/ respectively. Full hashes and paths are in the manifests and RESEARCH_LOG.md.

- **v1** (up to 128 probes): run ID 2026-09-24-probe-calibration-v1; commit e33f696; Slurm 22348.
- **v2** (up to 1024 probes; 3x160 power): run ID 2026-09-24-probe-calibration-v2; commit 12bed935; Slurm 22351.
- **Initialization ablation:** run ID 2026-09-24-probe-calibration-initialization; commit 151934d; Slurm 22355.
- **ChebNet replication:** run ID 2026-09-24-probe-calibration-chebnet; commit 1224c6b; Slurm 22359.
- **Pierre projected estimator:** run ID 2026-09-24-pierre-estimator-calibration; calibration commit 3aab845; Slurm 22377; curation Slurm 22380.
- **Estimator wall-clock:** run ID 2026-09-24-estimator-wall-clock; commit 8384aec; validation Slurm 22387; benchmark Slurm 22388.
- **Test-dataset overview:** run ID 2026-09-24-test-dataset-overview; final plotting Slurm 22390.
- **Lyapunov-rate histograms:** per-model runs 2026-09-24-lyapunov-histogram-{gcn, chebnet, sheaf} (900 draws each, Slurm 22400) compared by curation (Slurm 22402) into evidence/2026-09-24-lyapunov-histogram-comparison/. Checkpoints trained offline (jobs 22398/22399) under checkpoints/ (ignored, hashes in manifests).

All runs used Python 3.12.3 and PyTorch 2.14.0+cu130 in float64.

Lockfile SHA-256: f1fc70f6db931bcdfffb94d353d7ff9e70ed1bcb93899b0efde5668599f7c470e. Curation jobs were 22352/22353/22356. Full manifests state seeds, selected hyperparameters, tensor dimensions, training metrics, power residuals and environment. RESEARCH_LOG.md holds command/log paths and prior validation evidence.

References

Local-source provenance is indexed in papers/README.md.

- Kipf & Welling, Semi-Supervised Classification with Graph Convolutional Networks, arXiv:1609.02907.
- Defferrard, Bresson & Vandergheynst, Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering, arXiv:1606.09375.

- Bodnar et al., Neural Sheaf Diffusion, arXiv:2202.04579.
- Oono & Suzuki, Graph Neural Networks Exponentially Lose Expressive Power for Node Classification, arXiv:1905.10947. Its contraction result concerns distance to an invariant/smoothed subspace under assumptions; it is not a license to insert P into our estimator.
- Johansson, Strannegård, Engsner & Mostad, Exact Spectral Norm Regularization for Neural Networks, arXiv:2206.13581. Matrix-free JVP/VJP power route for a Jacobian spectral norm.
- Cui, Yan, Liu & Lu, Generalizing and Improving Jacobian and Hessian Regularization, arXiv:2212.00311. Matrix-free Lanczos motivation for Jacobian extremal norms.
- Kuczyński & Woźniakowski, Estimating the Largest Eigenvalue by the Power and Lanczos Algorithms with a Random Start, SIAM J. Matrix Anal. Appl. 13(4), 1992, DOI:10.1137/0613066. Random-start and Krylov convergence dependence.
- Benettin et al., Lyapunov Characteristic Exponents for Smooth Dynamical Systems, Meccanica 15, 1980. QR/orthonormal-frame context for asymptotic spectra.